

LLM Deployment and Industry Applications: Memory, Latency, Cost and Operations

Unit 10 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): what deployment is about, and a real model	5
3.2 Tier 2 (medium): the KV cache and grouped-query attention	6
3.3 Tier 3 (medium-hard): speed, batching, percentiles	7
3.3.1 Decoding is memory-bound: the roofline model	7
3.3.2 Batching policies	8
3.3.3 Latency percentiles and a real service	9
3.4 Tier 4 (hard): cost, operations, industry	10
3.4.1 Cost per token, rent or pay per token, cascades	10
3.4.2 LLMOps: monitoring and drift	10
3.4.3 Scaling and cloud versus local	11
3.4.4 Industry applications	11
3.5 Tier 5 (hardest): open questions	12
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures: deploy a RAG application as a web service	13
5. Analyze	13

5.1 Limits	13
5.2 Which term dominates	14
5.3 Sensitivity	14
5.4 Failure modes	14
5.5 Uncertainty	14
6. Evaluate	14
7. Create	15
8. Visualization lesson	15
9. Problem set	16
10. Retrieval practice and recap	16
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. compute a model's **parameter count**, **weight memory** and **KV-cache memory** from its configuration file, and check the result against an independent source;
2. explain why **decoding is memory-bound** and prefill compute-bound, find the **batch size** where the limit changes, and say what **quantization** and **grouped-query attention** buy;
3. compare **static and continuous batching** and read **latency percentiles** and throughput;
4. estimate **cost per token**, the **break-even** between a rented GPU and a pay-per-token service, and the effect of a **model cascade**;
5. **deploy** an LLM/RAG application as a web service with validation and monitoring, measure it, and describe the **LLMOps** and **industry** issues around it.

- **Prerequisites:** Unit 2 (the Transformer block, attention and the quadratic cost, $12d^2$ parameters per block), Unit 3 (next-token decoding, the context window), Unit 4 (cost of conversation history), Unit 5 (the RAG system), Unit 6 (quantization and its memory arithmetic), Unit 9 (evaluation and drift of metrics).
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 6 figures, the module `codes/serving/serving_np` / (parameter and KV-cache arithmetic from a **real model configuration**, a roofline latency

model, a batching simulation, cost formulas, drift monitoring, a **FastAPI RAG web service** and a benchmark that starts it; 9 tests), the *LLM serving calculator* and *Serving simulator* widgets, and a Colab notebook for serving a real model.

- **Source anchors:** the **configuration** of Qwen/Qwen2.5-0.5B-Instruct and the **parameter count the hub reports for it** (494,032,768), both fetched from the Hugging Face hub on the day of writing and stored in the module; Kwon et al. 2023, arXiv:2309.06180 (PagedAttention and vLLM: “the key-value cache (KV cache) memory for each request is huge and grows and shrinks dynamically”, wasted by “fragmentation and redundant duplication, limiting the batch size”; vLLM “improves the throughput of popular LLMs by 2-4× with the same level of latency” over the systems they compare with); Vaswani et al. 2017, arXiv:1706.03762 (Unit 2). The arXiv ID and the quoted statements were checked in the PDF.
- **What is measured here, and what is not.** The **counts** (parameters, memory, KV cache) are exact arithmetic from a real configuration and are **verified against the hub**. The **web-service latency and throughput** are real measurements, but of a toy retrieval service on **this machine’s CPU**, and are machine-specific. The **GPU speed, batching and cost** results use a stated **model** (a roofline model with *illustrative* hardware numbers, a queue simulation, and *illustrative* prices); they show how the quantities relate, not the performance or price of any product. No language model was served on the machine that built this course; the notebook that serves one is **not executed**.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
inference, serving		running a trained model to answer requests	3.1	new
prefill and decode		process the prompt at once / generate one token at a time	3.1	new
KV cache		stored keys and values of all earlier tokens	3.2	new
grouped-query attention (GQA)	n_{kv}	several query heads share one key-value head	3.2	new
memory-bound, compute-bound		limited by reading memory / by arithmetic	3.3	new
roofline model		time = larger of memory time and compute time	3.3	new
batching (static, continuous)	B	decode several requests together; fixed batch vs slots that refill	3.3	new
latency, throughput, p95		time per request; tokens per second; the value 95% of requests beat	3.3	new
service-level objective (SLO)		a target such as “p95 below 2 s”	3.3	new
model cascade, routing		try a small model first, escalate hard cases	3.4	new
LLMOps, drift (PSI)		operating LLM systems; detecting a shift in inputs	3.4	new

Term	Symbol	One-line meaning	Where it appears	New / known
quantization		store weights in fewer bits	3.2	known
context window, tokens		the text the model reads at once	3.2	known

Terms *used but not taught here*: tensor and pipeline parallelism, speculative decoding, prefix caching, specific serving frameworks other than naming vLLM, container orchestration.

2. Remember

Two phases of inference

Prefill processes the whole prompt in one parallel pass and builds the **KV cache**: the keys and values of every prompt token, so that later tokens need not recompute them. **Decode** then generates tokens one at a time; each step reads all the weights and the whole KV cache, adds one token’s keys and values, and outputs one token.

Memory counts

Weights take (parameters $\times$ bytes per parameter). The KV cache takes, per token,

$$\text{KV bytes per token} = 2 \times L \times n_{kv} \times d_h \times b, \quad (1)$$

where 2 is for keys and values, L the number of layers, n_{kv} the number of key-value heads, d_h the head width and b the bytes per number.

Latency, throughput, percentiles, SLO

Latency is the time one request takes; **throughput** is the work done per second (requests or tokens). The **95th percentile** (p95) is the latency that 95% of requests are below; averages hide the slow tail that users notice. A **service-level objective** states a target such as “p95 under 2 seconds”.

Memory hooks.

- *Decoding is reading, not thinking*: every token requires reading the whole model from memory.
- *The cache is the hidden bill*: weights are paid once, the KV cache is paid by every user and every token.
- *Batching is a bus*: one trip of the weights can carry many passengers.
- *Measure the tail, not the average*.

Recall quiz (answers in Section 10).

1. What are the two phases of inference, and which one limits the speed at which a chat reply appears word by word?

2. How many bytes of KV cache does one token need for a model with 24 layers, 2 KV heads, head width 64, in 16-bit numbers?
3. Why does batching raise throughput almost for free at small batch sizes?
4. What does the 95th-percentile latency tell you that the average does not?
5. What was the first thing you check when a measured parameter count disagrees with a published one?

3. Understand: the derivation

3.1 Tier 1 (easy): what deployment is about, and a real model

Training a model is done once; **serving** it happens for every user, all day, and it is where most of the money and most of the failures are. Three resources decide whether a deployment works: **memory** (does it fit, and how many users?), **time** (how long does a reply take?) and **money** (what does a million tokens cost?). A fourth, **operations**, keeps it working: monitoring, updating, and handling the unexpected.

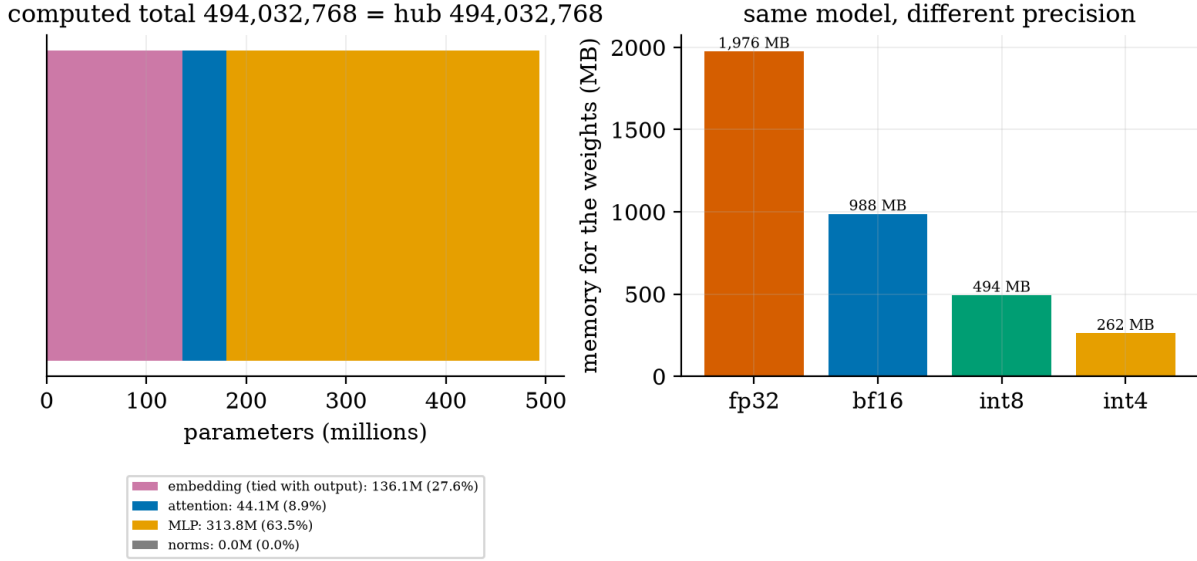
To make this concrete I use one **real model**: Qwen/Qwen2.5-0.5B-Instruct, whose `config.json` I fetched from the Hugging Face hub. Its numbers are: hidden width 896, 24 layers, 14 attention heads of width 64, **2 key-value heads**, feed-forward width 4864, vocabulary 151,936, tied input and output embeddings, 16-bit (bfloat16) weights, and a maximum context of 32,768 tokens.

From these alone I can compute how many parameters it has, following the architecture of Unit 2 (with a gated feed-forward layer of three matrices and attention biases on the query, key and value projections, which this model family uses):

the parameter count computed from the configuration (per layer: 14,912,384). **The Hugging Face hub reports exactly 494,032,768 parameters for this model: the difference is 0.** Checking a computation against an independent source is the habit to keep: if the numbers had disagreed, I would have looked first for a missing bias, an untied embedding or a different feed-forward shape. The size of the weights file agrees too: it is 988,097,824 bytes, which is $2 \times 494,032,768$ (two bytes per 16-bit weight) plus 32,288 bytes of file header.

part	formula	parameters	share
token embedding (tied with the output layer)	$V \cdot d = 151936 \cdot 896$	136,134,656	27.6%
attention, 24 layers	$L [dhd_h + hd_h + 2(dn_{kv}d_h + n_{kv}d_h) + hd_hd]$	44,067,840	8.9%
feed-forward (MLP), 24 layers	$L \cdot 3dd_{ff}$	313,786,368	63.5%
normalisation weights	$L \cdot 2d + d$	43,904	0.0%
total		494,032,768	

Notice that for this small model the embedding table is over a quarter of everything (the vocabulary is large); for large models the feed-forward layers dominate even more (the $12Ld^2$ of Unit 3).



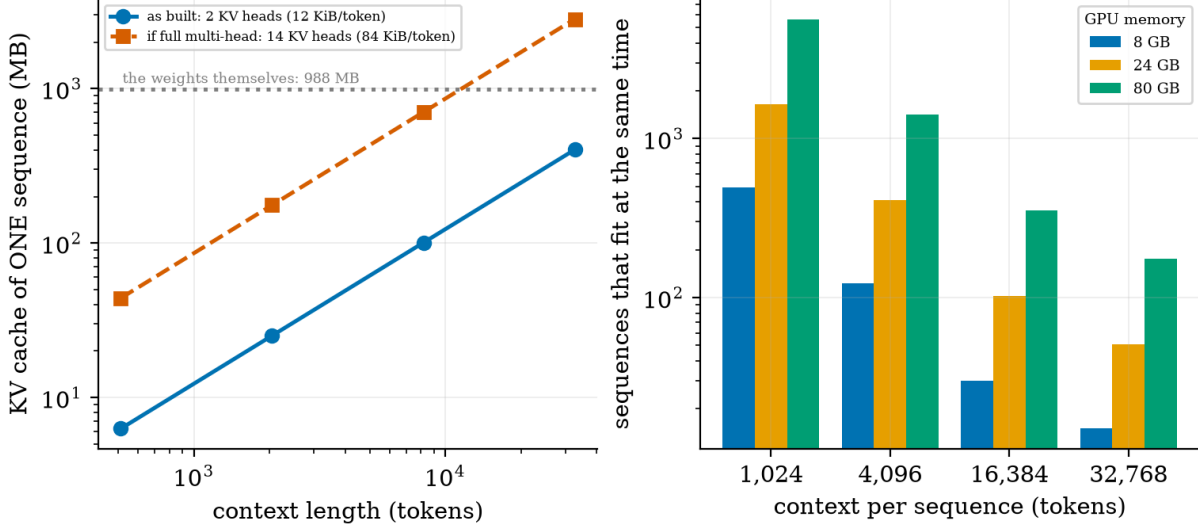
Left: where the 494 million parameters of Qwen2.5-0.5B are (embedding 27.6%, attention 8.9%, MLP 63.5%, norms 0.01%), with the computed total equal to the hub’s. Right: the memory for the weights at 32-bit (1976 MB), 16-bit (988 MB), 8-bit (494 MB) and 4-bit with one 16-bit scale per 64 weights (262.5 MB, 0.53125 bytes per weight, Unit 6).

3.2 Tier 2 (medium): the KV cache and grouped-query attention

At each decode step, the new token attends to all earlier tokens (Unit 2). Recomputing their keys and values every time would cost the whole prompt again, so they are **cached**. Per token, (1) gives for this model $2 \cdot 24 \cdot 2 \cdot 64 \cdot 2 = 12,288$ bytes (12 KiB). Multiply by the context length and the number of users to get the cache.

Grouped-query attention. The model has 14 query heads but only 2 key-value heads: several query heads *share* one key-value head. If it had one key-value head per query head (standard multi-head attention) the cache per token would be $2 \cdot 24 \cdot 14 \cdot 64 \cdot 2 = 86,016$ bytes: **7 times larger**. A single sequence of the full 32,768 tokens needs 402.7 MB of cache with grouped-query attention, against 2.82 GB without it.

How many users fit? With 16-bit weights and 10% of memory kept as overhead, an 8 GB GPU holds 493 sequences of 1024 tokens, 123 of 4096, 30 of 16,384 and 15 of 32,768 (without grouped-query attention: 70, 17, 4 and 2). On 24 GB: 1638, 409, 102 and 51; on 80 GB: 5643, 1410, 352 and 176. **Memory, not arithmetic, usually limits how many users a GPU can serve**, and the cache grows with *every* user and *every* token. Kwon et al. (2023) state the problem plainly: the KV cache memory “for each request is huge and grows and shrinks dynamically”, wasted by “fragmentation and redundant duplication, limiting the batch size”; their PagedAttention stores the cache in fixed-size blocks like virtual-memory pages, and vLLM “improves the throughput of popular LLMs by 2-4× with the same level of latency” compared with the systems they evaluated.



Left: KV cache of one sequence against its context length (log-log), as built (2 KV heads, 12 KiB per token, blue) and if it had one KV head per query head (14 heads, 84 KiB per token, red). The dotted line is the 988 MB of the weights: the full multi-head cache would exceed it after about 11,500 tokens, the grouped-query cache only after about 80,000 tokens ($988 \times 10^6 / 12288$), beyond this model's limit of 32,768. Right: how many sequences fit in 8, 24 and 80 GB of GPU memory (after the weights and a 10% overhead) at several context lengths.

3.3 Tier 3 (medium-hard): speed, batching, percentiles

3.3.1 Decoding is memory-bound: the roofline model

One decode step must (a) **read** every weight and every cached key and value from memory, and (b) **compute** about $2N$ floating-point operations per sequence for N parameters (one multiply and one add per weight). The step takes at least the longer of the two:

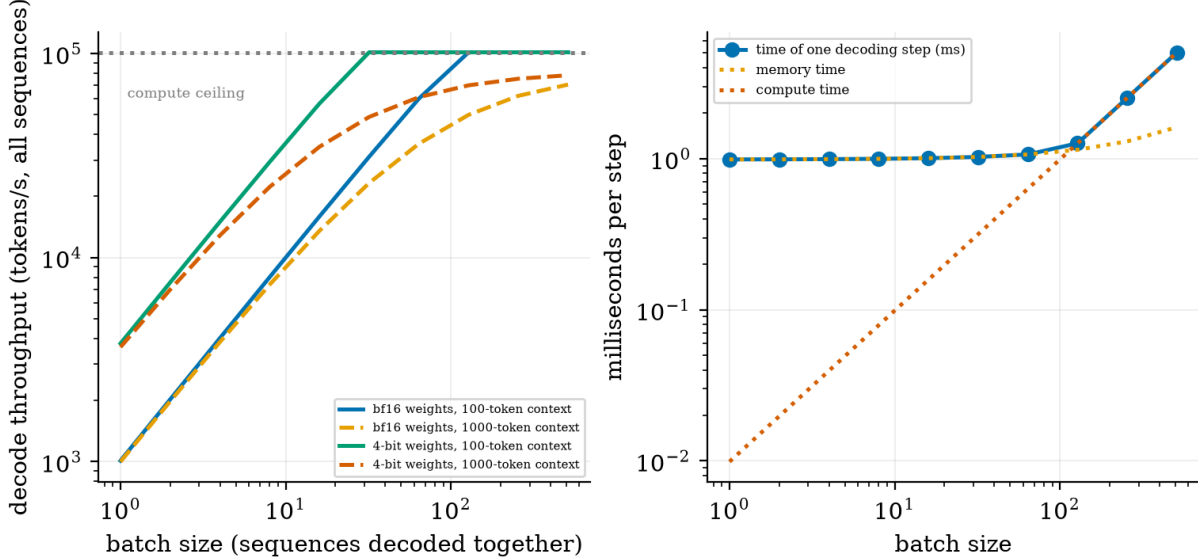
$$t_{\text{step}}(B) = \max \left(\underbrace{\frac{W + BK}{BW}}_{\text{memory time}}, \underbrace{\frac{B \cdot 2N}{F}}_{\text{compute time}} \right), \quad (2)$$

for a batch of B sequences, W bytes of weights, K bytes of cache per sequence, memory bandwidth BW (bytes/s) and peak arithmetic rate F (FLOP/s). The memory term contains the weights **once**, whatever B is: that is why batching is almost free. I evaluate it with *illustrative* hardware: $BW = 1$ TB/s and $F = 100$ TFLOP/s (round numbers chosen to show the pattern; they are not a product specification).

For the real model: $W = 988$ MB gives a memory time of 0.988 ms per step, so at $B = 1$ about **1000 tokens/s**. Compute per token is $2N/F = 9.88 \mu\text{s}$. The cache read per sequence is $1.23 \mu\text{s}$ for a 100-token context and $12.3 \mu\text{s}$ for 1000 tokens. Setting the two terms equal gives the **critical batch size**

$$B^* = \frac{W/BW}{2N/F - K/BW} \quad (\text{infinite if the denominator is } \leq 0). \quad (3)$$

For a 100-token context $B^* = 0.988 \text{ ms} / (9.88 - 1.23) \mu\text{s} = 114$: below it decoding is memory-bound and the throughput grows almost linearly with the batch; above it, compute-bound at the ceiling $F/(2N) = 101,208$ tokens/s. For a 1000-token context the cache read per sequence ($12.3 \mu\text{s}$) is **larger** than the compute per token ($9.9 \mu\text{s}$), so $B^* = \infty$: decoding stays memory-bound at every batch size.



Left: decode throughput (tokens/s summed over the batch) against batch size for the real model on illustrative hardware (1 TB/s, 100 TFLOP/s): 16-bit weights with 100-token contexts (solid blue; 1011, 8017, 31,147, 101,208 tokens/s at batches 1, 8, 32, 128, then the compute ceiling), 16-bit with 1000-token contexts (orange dashed; 1000, 7364, 23,167, 49,982), and 4-bit weights (green and red; 3792 tokens/s at batch 1). Right: the memory time, compute time and their maximum (the step time) against batch size for 100-token contexts.

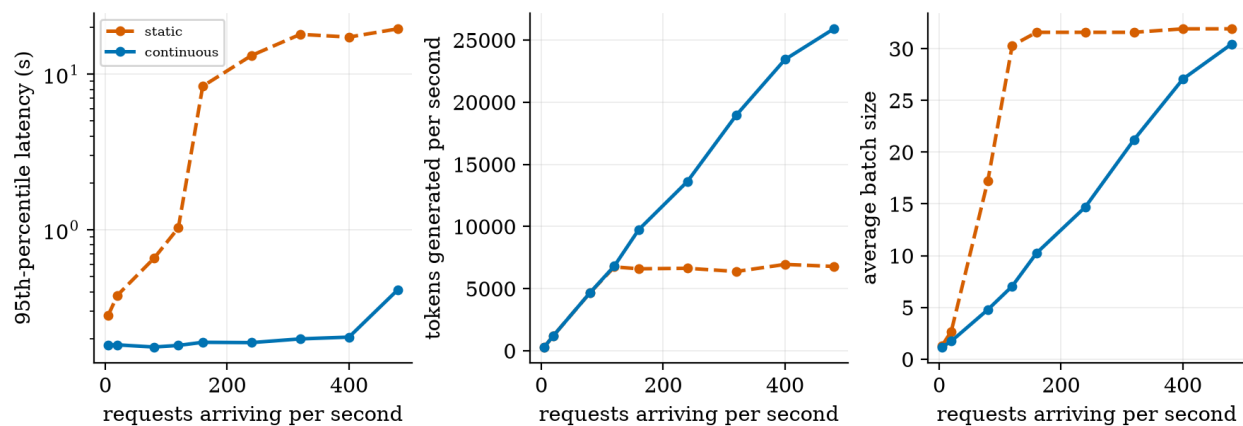
Four consequences:

1. **Quantization speeds up decoding, not only shrinks it.** With 4-bit weights (262.5 MB) the memory time is 0.263 ms: 3792 tokens/s at batch 1, **3.75 times** the 16-bit speed (Unit 6's arithmetic, now seen as speed).
2. **Prefill is different.** It processes many tokens at once, so it is compute-bound: a 1000-token prompt costs about $2N \cdot 1000 / (0.5F) = 19.8$ ms at an assumed 50% of peak arithmetic, against 0.99 ms for one decode step. This is why the *time to the first token* depends on the prompt length and the speed afterwards on the memory.
3. **Long contexts hurt twice:** they enlarge the cache (Figure 2) and keep decoding memory-bound.
4. **CPU inference** is the same equation with a smaller bandwidth: at an assumed 50 GB/s the 16-bit model needs 988 MB/50 GB/s = 19.8 ms per token (about 50 tokens/s) and the 4-bit one 5.2 ms (about 190 tokens/s): usable for a 0.5B model, hopeless for a 70B one.

3.3.2 Batching policies

If requests are processed one at a time the weights are re-read for each token. **Static batching** gathers up to $B_{\max}$ requests, runs them together until the *longest* finishes, then starts the next batch: short requests wait for long ones, and the GPU idles on finished slots. **Continuous batching** (iteration-level scheduling, used by systems such as vLLM) removes a finished request at once and lets a waiting one take its slot at the next step.

I simulated one replica of the real model: requests arrive as a Poisson process, each needs a random number of output tokens (geometric, mean 60), the step time is (2) with the real model's numbers (0.994 ms at batch 1, 1.037 at batch 8, 1.185 at batch 32), the largest batch is 32, and a static batch waits at most 50 ms to fill. (Prefill and KV memory limits are not simulated.)



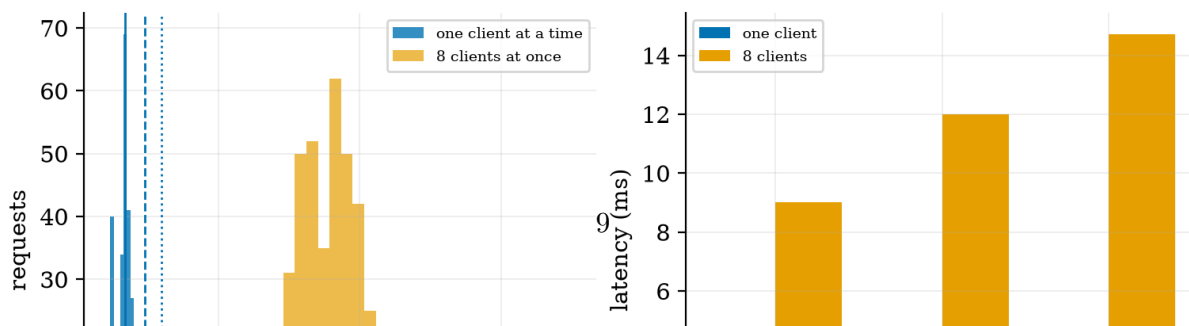
Simulation of static (red dashed) and continuous (blue) batching against the number of requests arriving per second: 95th-percentile latency (left, log scale), tokens generated per second (middle) and average batch size (right).

selected points of the simulation. Static batching reaches its capacity at about 6700 tokens/s (from 120 requests per second on, extra load only lengthens the queue: p95 goes from 1.0 s to 8.4 s at 160 requests per second); continuous batching keeps the p95 near 0.18 to 0.20 s up to 400 requests per second and still produces more tokens as load rises (25,902 tokens/s at 480), because a batch is only as slow as its longest member for static batching. The median (typical) wait is 0.04 s for continuous batching at all loads up to 400 requests/s against 0.11 to 9.5 s for static. These numbers come from a model with illustrative hardware; the *direction* (continuous batching is far better at high load, equal at low load) agrees with the 2-4 times throughput improvement that Kwon et al. report for their system, which also manages memory better.

requests per second	5	20	80	120	160	320	480
static: p95 latency (s)	0.28	0.38	0.66	1.03	8.38	18.0	19.6
continuous: p95 latency (s)	0.18	0.18	0.18	0.18	0.19	0.20	0.41
static: tokens/s	300	1212	4660	6747	6594	6379	6785
continuous: tokens/s	300	1212	4667	6821	9744	18,952	25,902

3.3.3 Latency percentiles and a real service

I deployed the course's RAG system (Unit 5: BM25 retrieval and an extractive answerer, with the abstention rule of Unit 5) as a web service with FastAPI, and benchmarked it by starting the server in a subprocess. The service has a health check, **validates input before doing any work** (a question must have 3 to 500 characters and k must be 1 to 8) and **logs every request**.



second. With 8 clients at once the throughput rose only to 864 requests per second (1.5 times) while the latency rose about 5 times (p50 9.0 ms, p95 12.0 ms, p99 14.7 ms): the single worker process is already busy, so extra clients queue. The three invalid requests (too short, too long, $k = 99$) were all rejected with HTTP status 422 before any retrieval. Of the 50 questions cycled (30 answerable, 10 off-topic, 10 near-miss), 70% were answered and the rest abstained. Time spent *inside* the handler was about 0.4 to 3 ms (from the service log) against 1.7 ms end to end for a typical request: the web layer adds its own overhead, so measure from the client. These numbers belong to this machine and this toy service.

An **SLO** turns such measurements into a promise: for example “p95 below 5 ms at 500 requests per second”. `slo_violations(latencies, 1.0)` returns the share of requests above a limit; tracking it over time is the first monitoring metric.

3.4 Tier 4 (hard): cost, operations, industry

3.4.1 Cost per token, rent or pay per token, cascades

With a GPU that costs c dollars per hour and produces τ tokens per second while busy a fraction u of the time, the cost of a million tokens is

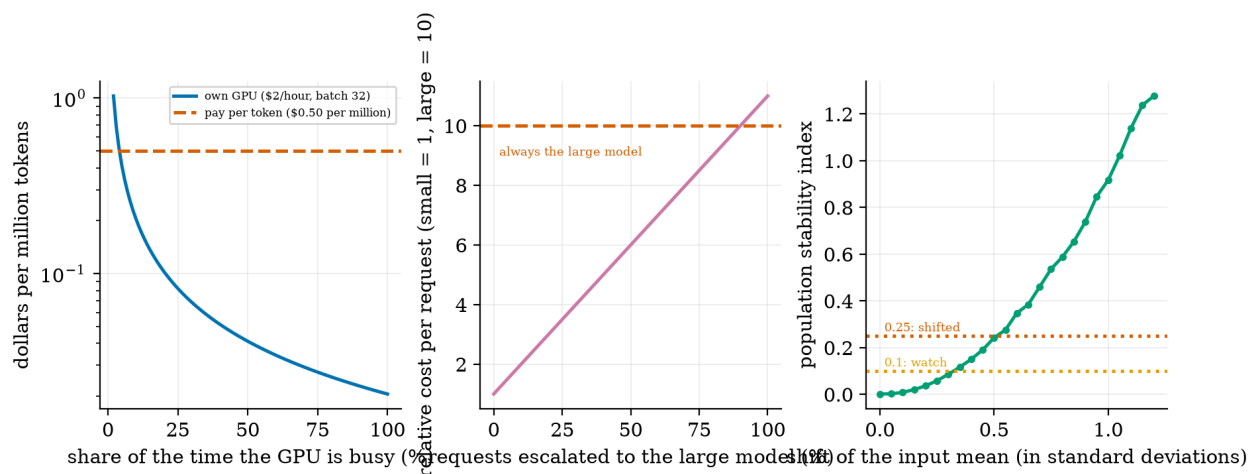
$$\text{cost per } 10^6 \text{ tokens} = \frac{c}{\tau u \cdot 3600} \times 10^6. \quad (4)$$

I use *illustrative* numbers: $c = \$2$ per hour and the batch-32 decode speed of the model on the illustrative hardware, $\tau = 27,012$ tokens/s. Then a million tokens cost **\$0.021 at 100% busy, \$0.041 at 50%, \$0.103 at 20%: utilisation is the lever**, and an idle GPU costs the same as a busy one. Against an illustrative pay-per-token price of \$0.50 per million tokens, renting the GPU is cheaper once it is busy more than 4.1% of the time; with a fixed monthly GPU cost of \$1460, the break-even volume is $1460/0.50 \times 10^6 = 2.92$ billion tokens per month. These numbers ignore prefill, engineering time, redundancy for failures and the hours when traffic is low, so real costs are higher; use them to see the structure (fixed cost against usage), not as a quote.

A **model cascade** sends every request to a small model and escalates a fraction p to a large one: expected cost $c_s + p c_l$. If the large model costs 10 times the small one: $p = 10\%$ gives 2.0, 20% gives 3.0, 50% gives 6.0 (relative to the small model alone), against 10 for always using the large one. It pays only if the *escalation rule is good* (it must detect the hard cases), which has to be measured with the methods of Unit 9. **Caching** (reusing the answer or the computation for repeated prefixes) and **quantization** (Section 3.3.1) are the other main cost levers.

3.4.2 LLMOps: monitoring and drift

A deployed system must be watched. Useful signals: **latency percentiles and error rates** (the SLO); **token and cost counters** per request; **quality signals** (Unit 9’s metrics on a sample of real requests, user feedback, abstention rate, groundedness); and **input drift**: a change in what users ask. The **population stability index** compares a reference distribution with a new one using bins from the reference quantiles: $\text{PSI} = \sum_i (a_i - e_i) \ln(a_i/e_i)$. On Gaussian data it was 0.001 for no shift, 0.085 for a shift of the mean by 0.3 standard deviations, 0.237 for 0.5 and 0.931 for a full standard deviation. The reading “below 0.1 stable, 0.1 to 0.25 shifting, above 0.25 shifted” is a common rule of thumb, not a law. Other LLMOps practice: **versioning** of the model, the prompt template, the index and the evaluation set; **gated releases** (a new version must pass the evaluation of Unit 9 before it serves



Left: cost per million tokens against the share of time the GPU is busy (illustrative: \$2 per hour, batch-32 decoding) and an illustrative \$0.50 pay-per-token price. Middle: relative cost of a cascade against the share of requests escalated to a model 10 times more expensive. Right: the population stability index against the shift of the input mean (in standard deviations), with the rule-of-thumb levels 0.1 and 0.25.

traffic, then a canary share of users); **rollback**; and **logging** with privacy controls (Unit 9: logs may contain personal data).

3.4.3 Scaling and cloud versus local

To serve more traffic: add replicas behind a load balancer (the web service above shows why: one worker saturates), scale them automatically with the queue length, and use batching inside each replica. **Cloud** services need no hardware and scale instantly but cost per token, send data off-site and depend on the vendor. **Local or private deployment** keeps data in-house and gives control, at the price of hardware, operations skills and low utilisation at night (the cost lesson above). Models small enough to run on a laptop (the 0.5B model needs 0.99 GB, or 0.26 GB at 4 bits) make a *local* option real for many tasks. The choice is about data sensitivity, volume, latency needs and skills, not only price.

3.4.4 Industry applications

where the earlier units apply. I list risks and controls, not statistics: the benefits and failure rates depend on the model and the task and are not measured here. In every row the pattern is the one of this course: **retrieve evidence (Unit 5)**, **keep the model's actions limited and validated (Unit 8)**, **evaluate on your own data with intervals (Unit 9)**, and **keep a person accountable**.

field	typical use	main risk	what to measure and control
healthcare	summarising notes, drafting letters, literature search	wrong or invented clinical facts; privacy	groundedness against the record, human sign-off, privacy controls, no autonomous advice

field	typical use	main risk	what to measure and control
finance	document analysis, report drafting, customer questions	wrong numbers; regulation; leakage	number checks (Unit 9), audit logs, approval for actions, access control
education	tutoring, practice questions, feedback	wrong explanations; over-reliance; fairness across students	accuracy on a reference set, per-group quality, teacher oversight
software engineering	code completion, review, test generation	insecure or wrong code; leaking secrets	tests that run, security review, no secrets in prompts
customer support	answering from a knowledge base, routing tickets	hallucinated policy; tone; escalation failures	groundedness, abstention and escalation rate, satisfaction, SLOs
legal and knowledge systems	search over contracts and case law, drafting	invented citations; confidentiality	citations that resolve, retrieval recall (Unit 5), confidentiality, lawyer review

3.5 Tier 5 (hardest): open questions

1. **How should capacity be planned for unpredictable demand?** Traffic is bursty; headroom costs money and queues cost users. Autoscaling reacts with a delay (starting a model takes time to load weights, see the 988 MB above).
2. **How much quality is lost by cheaper serving?** Quantization, smaller models and cascades save money; their effect on *your* task must be measured, and the loss can be uneven across inputs and groups (Unit 9).
3. **How to evaluate and monitor open-ended outputs in production?** Drift in inputs is easy to measure (PSI); drift in *quality* needs labelled samples or model judges whose own bias must be controlled (Unit 9).
4. **Efficiency research.** Faster decoding (speculative decoding, better cache layouts) and smaller caches are active areas; I did not test them.

4. Apply

4.1 Worked problems

Problem 1 (KV cache). 24 layers, 2 KV heads, head width 64, 16-bit: $2 \cdot 24 \cdot 2 \cdot 64 \cdot 2 = 12,288$ bytes per token; a 4096-token sequence: 50.3 MB.

Problem 2 (memory). A 6-billion-parameter model in 16-bit needs 12 GB for weights; in 4-bit with a scale per 64 weights, $6 \times 10^9 \times 0.53125 = 3.19$ GB. On an 8 GB GPU the first does not fit; the second leaves room for the cache.

Problem 3 (decode speed). Weights of 988 MB at 1 TB/s: 0.988 ms per step, about 1012 tokens/s at batch 1 (the roofline gives 1011 with the small cache read included).

Problem 4 (critical batch). $W = 988$ MB, $2N/F = 9.88 \mu\text{s}$, cache read $1.23 \mu\text{s}$ per sequence: $B^* = 988/(9.88 - 1.23) = 114$.

Problem 5 (cost). \$2 per hour, 27,012 tokens/s, 20% busy: $2/(27012 \cdot 0.2 \cdot 3600) \times 10^6 = \0.103 per million tokens.

Problem 6 (SLO). If the p95 is 12.0 ms and the target is 10 ms, more than 5% of requests violate it.

4.2 Code and figures: deploy a RAG application as a web service

Everything is in `codes/serving/serving_np/` (see its README). The service (`app_fastapi.py`):

```
class Ask(BaseModel):
    question: str = Field(min_length=3, max_length=500)    # validated before any work is
    done
    k: int = Field(default=3, ge=1, le=8)

@app.post("/ask")
def ask(req: Ask):
    order = R.rank(_bm25.scores(req.question))[: req.k]    # retrieve (Unit 5)
    ctx = [_chunks[int(i)]["text"] for i in order]
    cov = R.query_coverage(req.question, ctx[0])           # abstain when the
    evidence is weak
    answer = R.extractive_answer(req.question, ctx, n=2) if cov >= MIN_COVERAGE else None
    log.info("ask k=%d coverage=%.2f answered=%s latency_ms=%.2f", ...) # monitoring
    return {"answer": answer or "I don't know ...", "sources": [...], "latency_ms": ...}
```

Run it with `uvicorn app_fastapi:app --port 8000` and measure it with `python bench_service.py`. To serve a **real language model** instead of the extractive answerer, replace `answer_fn` by a call to your model, keep the validation, the abstention rule and the logging, and re-measure p50, p95 and throughput. The Colab notebook `u10_serve_real_model.ipynb` loads Qwen2.5-0.5B-Instruct, **recomputes its parameter count and KV cache from the loaded configuration**, measures tokens per second at several batch sizes and compares them with the roofline prediction. It was **not executed** while writing this document.

Try it in the playground

The **LLM serving calculator** computes memory, users that fit, decode speed and cost for the real Qwen configuration or an illustrative 6B shape (JavaScript checked to give the same counts as the Python code). The **Serving simulator** runs the static versus continuous batching comparison of Section 3.3.2 with your load, maximum batch and reply length.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), uncertainty (5.5).

5.1 Limits

- $B = 1$: decode time is W/BW : about 1 ms for the 16-bit model on 1 TB/s.

- $B \rightarrow \infty$: throughput approaches $F/(2N)$ if the cache read per sequence is smaller than the compute per token; otherwise the memory-bound limit BW/K tokens/s.
- Utilisation $u \rightarrow 0$: cost per token $\rightarrow \infty$.
- All requests of equal length: static batching loses its disadvantage.

5.2 Which term dominates

For short contexts and small batches the **weights** dominate memory time; for long contexts and many users the **KV cache** does (at 1000 tokens the cache read per sequence exceeds the compute per token). For cost, **utilisation** dominates the price per token.

5.3 Sensitivity

Decode speed is directly proportional to bandwidth and inversely to bytes per weight (quantization: $3.75\times$). Memory capacity in users is proportional to $1/(\text{context} \cdot n_{kv})$. Latency percentiles under load are very sensitive to the batching policy (Figure 4) and to the number of worker processes (Figure 5).

5.4 Failure modes

- **Out of memory** when the cache of all users exceeds the GPU.
- **Queues and a bad tail** under bursty load.
- **Silent quality loss** after quantization or a model swap.
- **Drift** in inputs or data after launch.
- **Unvalidated input** reaching the model or tools.
- **Logs with personal data.**
- **Cost surprises** from low utilisation or long contexts.

5.5 Uncertainty

The parameter and memory counts are exact and verified. The service numbers are one machine and one toy workload. The roofline, batching and cost numbers depend entirely on the *illustrative* hardware and prices (1 TB/s, 100 TFLOP/s, \$2 per hour, \$0.50 per million tokens): change them and the numbers change, though the relationships (memory-bound decoding, the batching gap, utilisation as the cost lever) do not.

6. Evaluate

Options compared. *Hosted API*: no operations, pay per token, data leaves your control. *Rented GPU with an open model*: control and low cost per token at high utilisation, but you operate it. *On-device or CPU*: private and offline, limited to small or quantized models (Section 3.3.1). *Cascades and caching*: the cheapest to run but need a reliable decision rule.

Cost against quality. Every saving in Section 3.4.1 must be paid back by an evaluation that shows quality is preserved on your task.

When not to self-host. Low or highly variable volume (utilisation below about 5% in the illustrative numbers), no operations skills, or no data restrictions that justify the effort.

Wider context. Deployment turns a model into a service with obligations: uptime, privacy, fairness over time, accountability. The engineering of this unit (measure, validate, monitor, version, roll back) and the evaluation of Unit 9 belong together.

7. Create

What you can build from this

1. **Config calculator:** fetch the `config.json` of another open model and compute its parameters and KV cache with `param_breakdown` (check against the hub's count; if it disagrees, find out why).
2. **Capacity planner:** given a GPU memory, a model, a context and a target number of users, report whether it fits, and the decode speed per user.
3. **Latency lab:** add a second worker process to the service and measure how p95 changes with 8 clients.
4. **Cascade study:** build a small-then-large pipeline for the course RAG (extractive then a real model) and choose an escalation rule using the coverage score of Unit 5; report cost and quality with intervals.
5. **Drift monitor:** log the length of incoming questions, compare a day with a reference using `psi`, and alert above 0.25.
6. **Combine with Unit 9:** add the groundedness and abstention metrics of Unit 9 to the service log, and track them over time.

8. Visualization lesson

charts taught here. How to read a latency histogram: the mass near the left is the typical request; the **right tail** is what users complain about; compare percentile lines, not a mean.

Chart type	Best for	How it misleads	matplotlib call
stacked horizontal bar of parts	where a total comes from (Fig. 1)	tiny parts vanish: label the numbers	<code>ax.barh(0, v, left=left)</code>
log-log line with reference lines	quantities spanning orders of magnitude (Fig. 2, 3)	straight lines can hide constant factors	<code>ax.loglog(x, y); ax.axhline(...)</code>
three curves on one axis (memory, compute, maximum)	which limit applies (Fig. 3 right)	the crossover depends on the assumed hardware	<code>ax.loglog(b, mem); ax.loglog(b, comp)</code>
paired policy comparison across load	system behaviour under load (Fig. 4)	simulation, not measurement: state the model	<code>ax.semilogy(rate, p95)</code>
latency histogram with percentile lines	the shape of the tail (Fig. 5)	an average line would hide it	<code>ax.hist(...); ax.axvline(np.percentile(a, 95))</code>
cost curve with a price line	break-even (Fig. 6)	the assumptions are in the axes: print them	<code>ax.semilogy(u, cost); ax.axhline(price)</code>

9. Problem set

Graded exercises

1. (easy) A model has 32 layers, 8 key-value heads of width 128, 16-bit cache. Bytes of KV cache per token, and for a 2000-token sequence?
2. (medium) The same model has 7 billion parameters in 16-bit on a 24 GB GPU with 10% overhead: how many 2000-token sequences fit?
3. (medium-hard) Derive (3) from the two terms of (2). Evaluate it for $W = 14$ GB, $N = 7 \times 10^9$, $K = 0.26$ GB, $BW = 2$ TB/s, $F = 300$ TFLOP/s.
4. (hard) A rented GPU costs \$1500 per month. A provider charges \$0.80 per million tokens. Find the monthly volume at which they cost the same, and the average number of tokens per second this is.
5. (hardest) A cascade sends all requests to a small model (cost 1, accuracy 0.80) and escalates when a confidence score is below a threshold to a large model (cost 12, accuracy 0.95). Suppose the escalation rule sends 30% of requests, and among them the small model would have been right 40% of the time. Compute the expected cost and the expected accuracy, and compare with always using the large model.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. Prefill (the whole prompt at once) and decode (one token at a time); decode limits how fast the reply appears word by word.
2. $2 \cdot 24 \cdot 2 \cdot 64 \cdot 2 = 12,288$ bytes.
3. The weights are read once per step whatever the batch, so adding sequences to the batch adds little time until compute becomes the limit.
4. How slow the slowest 5% are: the tail users experience; an average hides it.
5. Look for a missing or extra component in your own count (biases, tied embeddings, feed-forward shape), then check the source.

Explain it to a friend.

- Why does a long chat make a GPU serve fewer people?
- Why is a 4-bit model not only smaller but faster at generating text?
- Why can a rented GPU be a worse deal than paying per token?

Re-derive tomorrow without looking: (a) KV bytes per token; (b) the critical batch (3); (c) the cost per million tokens (4).

11. Where next

This is the last unit of the course. The units fit together as one progression: **models** (Units 1 to 3), **using them** (Units 4 to 6), **extending them** (Units 7 and 8), **trusting them** (Unit 9) and **running**

them (this unit). A sensible next step is a capstone: build a retrieval-based assistant over your own documents (Units 5, 9, 10), evaluate it with intervals, deploy it as a monitored service, and write the model-card-style documentation of its limits.

- **Terms met but not taught:** speculative decoding, prefix caching, parallelism for very large models.
- **Open threads:** how real GPU measurements compare with the roofline predictions (the Colab notebook); how much quality quantization costs on your task.